INFORME DE GUÍA PRÁCTICA

T6. Implementación de un árbol binario en Java - Sistema de organización jerárquica de departamentos universitarios

I. PORTADA

Dato	Información
Unidad curricular	Profesional
Nivel y paralelo	3ro Software "B"
Modalidad	Individual
Estudiante	Villacres Lopez Walter Fernando
Asignatura	Estructura de Datos
Docente	Ing. José Caiza
Repositorio	https://github.com/fercho123909/T6-ArbolBinario-FernandoVillacres

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General

Implementé un árbol binario con programación orientada a objetos para insertar, consultar y analizar la jerarquía de dependencias universitarias.

2.2 Modalidad, recursos y actividades

Usé Java (JDK), Visual Studio Code, Git, GitHub, plataformas y aplicaciones educativas. La inteligencia artificial se empleó como apoyo. Implementé Nodo, las operaciones recursivas, el menú y los cinco casos obligatorios. La práctica fue individual.

2.3 Árbol inicial

```
UNIVERSIDAD
|-- [I] FISEI
|   |-- [I] SOFTWARE
|   |   |-- [I] NIVEL1
|   |   `-- [D] NIVEL3
|   `-- [D] TI
`-- [D] DIDE
    |-- [I] INVESTIGACIÓN
    `-- [D] VINCULACIÓN
```

El árbol tiene 9 nodos, 5 hojas y altura 3 en aristas. [I] representa hijo izquierdo y [D] hijo derecho.

Diseño, resultados y rúbrica

Organicé el programa en los paquetes modelo, negocio y app. Nodo guarda el dato y sus hijos; ArbolBinario valida inserciones y calcula recursivamente nodos, hojas y altura; Main usa Scanner y ofrece las opciones 1 a 9.

Nodo	Grado	Hoja	Hijos
UNIVERSIDAD	2	No	FISEI, DIDE
FISEI	2	No	SOFTWARE, TI
DIDE	2	No	INVESTIGACIÓN, VINCULACIÓN
SOFTWARE	2	No	NIVEL1, NIVEL3
TI, INVESTIGACIÓN, VINCULACIÓN, NIVEL1, NIVEL3	0	Sí	-

Casos obligatorios

Caso	Esperado	Obtenido
Árbol vacío	0 nodos; altura -1	0 nodos; altura -1
Un nodo	1 nodo; 1 hoja; altura 0	1 nodo; 1 hoja; altura 0
Árbol inicial	9 nodos; 5 hojas; altura 3	9 nodos; 5 hojas; altura 3
Inserción ocupada	Rechazada; estructura intacta	Rechazada; quedan 9 nodos
Grados	FISEI=2; SOFTWARE=2	FISEI=2; SOFTWARE=2

Complejidad

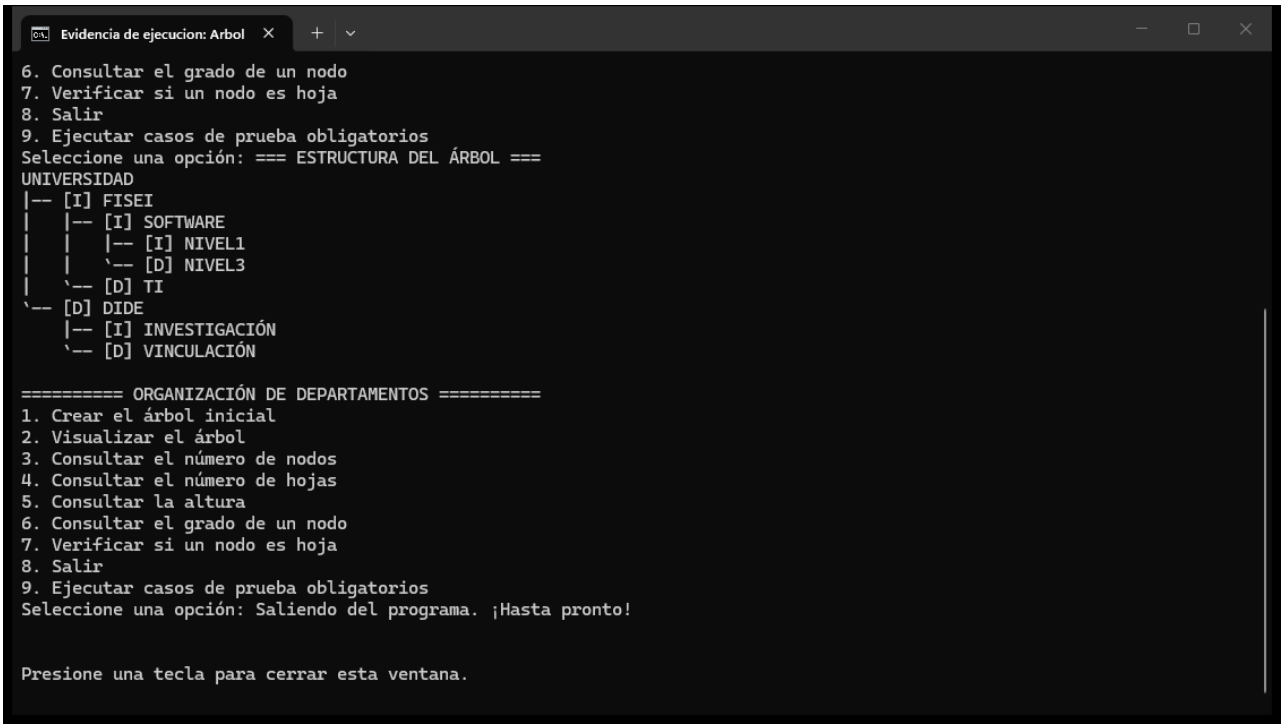
Operación	Tiempo	Espacio
contarNodos, contarHojas, altura	O(n)	O(h)
buscar	O(n) peor caso	O(h)
agregarIzquierdo/agregarDerecho	O(1)	O(1)
esVacio, esHoja, grado	O(1)	O(1)

n es el número de nodos y h la altura. La búsqueda es O(n) porque el árbol de departamentos no tiene un orden de búsqueda.

Compilación

```
javac -encoding UTF-8 -d out src/modelo/Nodo.java src/negocio/ArbolBinario.java src/app/Main.java
java -cp out app.Main
```

Evidencia 1. Menú y visualización



Captura real de la consola del programa mostrando el menú y el árbol con sus lados.

Evidencia 2. Casos de prueba

```
SALIDA REAL DE LAS PRUEBAS AUTOMATIZADAS

[Caso 1] Árbol vacío: esVacio=true; nodos=0; altura=-1
[Caso 2] Un nodo: nodos=1; hojas=1; altura=0
[Caso 3] Árbol inicial: nodos=9; hojas=5; altura=3
[Caso 4] Inserción ocupada: rechazada; nodos=9 sin cambios
[Caso 5] Grado: FISEI=2; SOFTWARE=2
Todas las pruebas pasaron correctamente.
```

Vista visual preparada desde la transcripción real de la ejecución; no es una captura directa de pantalla.

```
[Caso 1] Árbol vacío: esVacio=true; nodos=0; altura=-1
[Caso 2] Un nodo: nodos=1; hojas=1; altura=0
[Caso 3] Árbol inicial: nodos=9; hojas=5; altura=3
[Caso 4] Inserción ocupada: rechazada; nodos=9 sin cambios
[Caso 5] Grado: FISEI=2; SOFTWARE=2
Todas las pruebas pasaron correctamente.
```

Transcripción guardada directamente desde la ejecución del ejecutor automatizado.

Para agregar una captura genuina, guardo una imagen directa de la consola como evidencias/captura_casos.png y vuelvo a generar este PDF.

Reflexiones

1. La altura cuenta las aristas del camino más largo desde la raíz hasta una hoja. En mi árbol, UNIVERSIDAD → FISEI → SOFTWARE → NIVEL1 tiene 3 aristas; las ramas más cortas no cambian ese valor. En el método recursivo se compara la altura de ambos hijos con Math.max.

2. Una lista enlazada es lineal y para buscar suele recorrerse nodo por nodo, con costo O(n). Un árbol binario representa relaciones jerárquicas. Si está ordenado y equilibrado, una búsqueda puede ser O(log n); si se degenera en una rama, vuelve a O(n). El árbol de esta práctica no está ordenado, así que su búsqueda es O(n).

3. En SmartETL podría usar un árbol de decisión para validar registros, con cada condición enviando el dato por una rama. También se podría indexar por una clave con un árbol binario de búsqueda equilibrado, o representar expresiones de transformación. Para muchos registros convendría evaluar AVL o rojo-negro.

Conclusiones y recomendaciones

Separar modelo, negocio y app ayudó a organizar la solución. La recursividad simplificó los conteos y la altura, y las pruebas confirmaron los casos límite. Recomiendo probar siempre árbol vacío y unitario, validar inserciones y, para búsquedas masivas, evaluar árboles equilibrados.

Habilidades que practiqué

Marqué pensamiento crítico, adaptabilidad y responsabilidad durante el desarrollo de esta práctica.

Control de versiones

Commit	Cambio
72568b1	Documenta la prÃ¡ctica y agrega evidencias verificadas
2400e38	Actualiza informe final con historial de entrega
0c18974	Agrega visual de los casos de prueba al informe
29fc226	Adapta informe al formato institucional

Referencias

[1] T. H. Cormen et al., Introduction to Algorithms, 4.ª ed., MIT Press, 2022.
[2] M. A. Weiss, Data Structures and Algorithm Analysis in Java, 3.ª ed., Pearson, 2012.
[3] R. Sedgewick y K. Wayne, Algorithms, 4.ª ed., Addison-Wesley, 2011.
[4] Oracle, Java Platform, Standard Edition Documentation, https://docs.oracle.com/en/java/.
[5] F. Villacres, "T6-ArbolBinario-FernandoVillacres," GitHub, 2026. Repositorio.

Anexos: código fuente

Anexo 1. Clase Nodo<T>

```
package modelo;

public class Nodo<T> {
    private T dato;
    private Nodo<T> izquierdo;
    private Nodo<T> derecho;

    public Nodo(T dato) {
        this.dato = dato;
    }

    public T getDato() {
        return dato;
    }

    public void setDato(T dato) {
        this.dato = dato;
    }

    public Nodo<T> getIzquierdo() {
        return izquierdo;
    }

    public void setIzquierdo(Nodo<T> izquierdo) {
        this.izquierdo = izquierdo;
    }

    public Nodo<T> getDerecho() {
        return derecho;
    }

    public void setDerecho(Nodo<T> derecho) {
        this.derecho = derecho;
    }

    public boolean esHoja() {
        return izquierdo == null && derecho == null;
    }

    public int grado() {
        int cantidadHijos = 0;
        if (izquierdo != null) {
            cantidadHijos++;
        }
        if (derecho != null) {
            cantidadHijos++;
        }
        return cantidadHijos;
    }

    @Override
    public String toString() {
        return String.valueOf(dato);
    }
}
```

Anexo 2. Clase ArbolBinario<T>

```
package negocio;

import java.util.Objects;
import modelo.Nodo;

public class ArbolBinario<T> {
    private Nodo<T> raiz;

    public Nodo<T> crearRaiz(T dato) {
        if (raiz != null) {
            throw new IllegalStateException("El árbol ya tiene una raíz: " + raiz.getDato());
        }
        raiz = new Nodo<>(dato);
        return raiz;
    }

    public Nodo<T> agregarIzquierdo(Nodo<T> padre, T dato) {
        validarPadre(padre);
        if (padre.getIzquierdo() != null) {
            throw new IllegalStateException("La posición izquierda de '" + padre.getDato()
                    + "' ya está ocupada por '" + padre.getIzquierdo().getDato() + "'.");
        }
        Nodo<T> hijo = new Nodo<>(dato);
        padre.setIzquierdo(hijo);
        return hijo;
    }

    public Nodo<T> agregarDerecho(Nodo<T> padre, T dato) {
        validarPadre(padre);
        if (padre.getDerecho() != null) {
            throw new IllegalStateException("La posición derecha de '" + padre.getDato()
                    + "' ya está ocupada por '" + padre.getDerecho().getDato() + "'.");
        }
        Nodo<T> hijo = new Nodo<>(dato);
        padre.setDerecho(hijo);
        return hijo;
    }

    public Nodo<T> getRaiz() {
        return raiz;
    }

    public int contarNodos() {
        return contarNodos(raiz);
    }

    private int contarNodos(Nodo<T> nodo) {
        if (nodo == null) {
            return 0;
        }
        return 1 + contarNodos(nodo.getIzquierdo()) + contarNodos(nodo.getDerecho());
    }

    public int contarHojas() {
        return contarHojas(raiz);
    }

    private int contarHojas(Nodo<T> nodo) {
        if (nodo == null) {
            return 0;
        }
        if (nodo.esHoja()) {
            return 1;
        }
        return contarHojas(nodo.getIzquierdo()) + contarHojas(nodo.getDerecho());
    }

    public int altura() {
        return altura(raiz);
    }

    private int altura(Nodo<T> nodo) {
        if (nodo == null) {
            return -1;
        }
        return 1 + Math.max(altura(nodo.getIzquierdo()), altura(nodo.getDerecho()));
    }

    public boolean esVacio() {
        return raiz == null;
    }

    public Nodo<T> buscar(T dato) {
        return buscar(raiz, dato);
    }

    private Nodo<T> buscar(Nodo<T> nodo, T dato) {
        if (nodo == null) {
            return null;
        }
        if (Objects.equals(nodo.getDato(), dato)) {
            return nodo;
```

```
        }
        Nodo<T> encontrado = buscar(nodo.getIzquierdo(), dato);
        return encontrado != null ? encontrado : buscar(nodo.getDerecho(), dato);
    }

    public void mostrarArbol() {
        if (raiz == null) {
            System.out.println("(árbol vacío)");
            return;
        }
        System.out.println(raiz.getDato());
        if (raiz.getIzquierdo() != null) {
            mostrarRama(raiz.getIzquierdo(), "", raiz.getDerecho() == null, "I");
        }
        if (raiz.getDerecho() != null) {
            mostrarRama(raiz.getDerecho(), "", true, "D");
        }
    }

    private void mostrarRama(Nodo<T> nodo, String prefijo, boolean esUltimo, String lado) {
        System.out.println(prefijo + (esUltimo ? "`-- " : "|-- ") + "[" + lado + "] " + nodo.getDato());
        String prefijoHijos = prefijo + (esUltimo ? "    " : "|   ");
        if (nodo.getIzquierdo() != null) {
            mostrarRama(nodo.getIzquierdo(), prefijoHijos, nodo.getDerecho() == null, "I");
        }
        if (nodo.getDerecho() != null) {
            mostrarRama(nodo.getDerecho(), prefijoHijos, true, "D");
        }
    }

    private void validarPadre(Nodo<T> padre) {
        if (padre == null) {
            throw new IllegalArgumentException("El nodo padre no puede ser nulo.");
        }
    }
}
```

Anexo 3. Programa principal

```
package app;

import java.util.Scanner;
import modelo.Nodo;
import negocio.ArbolBinario;

public class Main {
    private static final Scanner SCANNER = new Scanner(System.in);
    private static ArbolBinario<String> arbol = new ArbolBinario<>();

    public static void main(String[] args) {
        boolean continuar = true;
        while (continuar) {
            mostrarMenu();
            int opcion = leerEntero("Seleccione una opción: ");
            switch (opcion) {
                case 1:
                    crearArbolInicial();
                    break;
                case 2:
                    System.out.println("=== ESTRUCTURA DEL ÁRBOL ===");
                    arbol.mostrarArbol();
                    break;
                case 3:
                    System.out.println("Cantidad de nodos: " + arbol.contarNodos());
                    break;
                case 4:
                    System.out.println("Cantidad de hojas: " + arbol.contarHojas());
                    break;
                case 5:
                    System.out.println("Altura en aristas: " + arbol.altura());
                    break;
                case 6:
                    consultarGrado();
                    break;
                case 7:
                    consultarHoja();
                    break;
                case 8:
                    System.out.println("Saliendo del programa. ¡Hasta pronto!");
                    continuar = false;
                    break;
                case 9:
                    ejecutarCasosDePrueba();
                    break;
                default:
                    System.out.println("Opción no válida. Elija un número del 1 al 9.");
            }
            System.out.println();
        }
    }

    private static void mostrarMenu() {
        System.out.println("========== ORGANIZACIÓN DE DEPARTAMENTOS ==========");
        System.out.println("1. Crear el árbol inicial");
        System.out.println("2. Visualizar el árbol");
        System.out.println("3. Consultar el número de nodos");
        System.out.println("4. Consultar el número de hojas");
        System.out.println("5. Consultar la altura");
        System.out.println("6. Consultar el grado de un nodo");
        System.out.println("7. Verificar si un nodo es hoja");
        System.out.println("8. Salir");
        System.out.println("9. Ejecutar casos de prueba obligatorios");
    }

    private static void crearArbolInicial() {
        arbol = construirArbolInicial();
        System.out.println("Árbol inicial creado correctamente.");
    }

    private static ArbolBinario<String> construirArbolInicial() {
        ArbolBinario<String> nuevoArbol = new ArbolBinario<>();
        Nodo<String> universidad = nuevoArbol.crearRaiz("UNIVERSIDAD");
        Nodo<String> fisei = nuevoArbol.agregarIzquierdo(universidad, "FISEI");
        Nodo<String> dide = nuevoArbol.agregarDerecho(universidad, "DIDE");
        Nodo<String> software = nuevoArbol.agregarIzquierdo(fisei, "SOFTWARE");
        nuevoArbol.agregarDerecho(fisei, "TI");
        nuevoArbol.agregarIzquierdo(dide, "INVESTIGACIÓN");
        nuevoArbol.agregarDerecho(dide, "VINCULACIÓN");
        nuevoArbol.agregarIzquierdo(software, "NIVEL1");
        nuevoArbol.agregarDerecho(software, "NIVEL3");
        return nuevoArbol;
    }

    private static void ejecutarCasosDePrueba() {
        System.out.println("=== CASOS DE PRUEBA OBLIGATORIOS ===");

        ArbolBinario<String> vacio = new ArbolBinario<>();
        System.out.println("[Caso 1] Árbol vacío");
        System.out.println("esVacio(): " + vacio.esVacio());
        System.out.println("nodos: " + vacio.contarNodos() + " (esperado 0)");
        System.out.println("altura: " + vacio.altura() + " (esperado -1)");
```

```
        ArbolBinario<String> unNodo = new ArbolBinario<>();
        unNodo.crearRaiz("PRUEBA");
        System.out.println("[Caso 2] Árbol con un solo nodo");
        System.out.println("nodos: " + unNodo.contarNodos() + " (esperado 1)");
        System.out.println("hojas: " + unNodo.contarHojas() + " (esperado 1)");
        System.out.println("altura: " + unNodo.altura() + " (esperado 0)");

        ArbolBinario<String> arbolPrueba = construirArbolInicial();
        System.out.println("[Caso 3] Árbol inicial");
        arbolPrueba.mostrarArbol();
        System.out.println("nodos: " + arbolPrueba.contarNodos() + " (esperado 9)");
        System.out.println("hojas: " + arbolPrueba.contarHojas() + " (esperado 5)");
        System.out.println("altura: " + arbolPrueba.altura() + " (esperado 3)");

        System.out.println("[Caso 4] Intento de insertar un hijo en una posición ocupada");
        Nodo<String> fisei = arbolPrueba.buscar("FISEI");
        int nodosAntes = arbolPrueba.contarNodos();
        try {
            arbolPrueba.agregarIzquierdo(fisei, "NUEVO");
            System.out.println("ERROR: la inserción debió ser rechazada.");
        } catch (IllegalStateException e) {
            System.out.println("Inserción rechazada: " + e.getMessage());
        }
        System.out.println("nodos tras el intento: " + arbolPrueba.contarNodos() + " (sin cambios: "
                + (nodosAntes == arbolPrueba.contarNodos()) + ")");

        System.out.println("[Caso 5] Grado de FISEI y SOFTWARE");
        System.out.println("grado(FISEI): " + fisei.grado() + " (esperado 2)");
        System.out.println("grado(SOFTWARE): " + arbolPrueba.buscar("SOFTWARE").grado() + " (esperado 2)");
    }

    private static void consultarGrado() {
        Nodo<String> nodo = buscarNodoSolicitado();
        if (nodo != null) {
            System.out.println("El grado de '" + nodo.getDato() + "' es: " + nodo.grado());
        }
    }

    private static void consultarHoja() {
        Nodo<String> nodo = buscarNodoSolicitado();
        if (nodo != null) {
            System.out.println("'" + nodo.getDato() + "' " + (nodo.esHoja() ? "SÍ" : "NO") + " es una hoja.");
        }
    }

    private static Nodo<String> buscarNodoSolicitado() {
        String nombre = leerTexto("Ingrese el nombre del nodo: ");
        Nodo<String> nodo = arbol.buscar(nombre);
        if (nodo == null) {
            System.out.println("No existe un nodo llamado '" + nombre + "'.");
        }
        return nodo;
    }

    private static int leerEntero(String mensaje) {
        while (true) {
            System.out.print(mensaje);
            String entrada = SCANNER.nextLine().trim();
            try {
                return Integer.parseInt(entrada);
            } catch (NumberFormatException e) {
                System.out.println("Ingrese un número entero válido.");
            }
        }
    }

    private static String leerTexto(String mensaje) {
        System.out.print(mensaje);
        return SCANNER.nextLine().trim();
    }
}
```

Anexo 4. Pruebas automatizadas

```
package test;

import modelo.Nodo;
import negocio.ArbolBinario;

public class PruebasArbolBinario {
    public static void main(String[] args) {
        probarArbolVacio();
        System.out.println("[Caso 1] Árbol vacío: esVacio=true; nodos=0; altura=-1");
        probarUnNodo();
        System.out.println("[Caso 2] Un nodo: nodos=1; hojas=1; altura=0");
        ArbolBinario<String> arbol = construirArbolUniversitario();
        probarArbolUniversitario(arbol);
        System.out.println("[Caso 3] Árbol inicial: nodos=9; hojas=5; altura=3");
        probarInsercionOcupada(arbol);
        System.out.println("[Caso 4] Inserción ocupada: rechazada; nodos=9 sin cambios");
        probarGrados(arbol);
        System.out.println("[Caso 5] Grado: FISEI=2; SOFTWARE=2");
        System.out.println("Todas las pruebas pasaron correctamente.");
    }

    private static void probarArbolVacio() {
        ArbolBinario<String> arbol = new ArbolBinario<>();
        verificar(arbol.esVacio(), "El árbol nuevo debe estar vacío.");
        verificar(arbol.contarNodos() == 0, "El árbol vacío debe tener cero nodos.");
        verificar(arbol.contarHojas() == 0, "El árbol vacío debe tener cero hojas.");
        verificar(arbol.altura() == -1, "La altura del árbol vacío debe ser -1.");
    }

    private static void probarUnNodo() {
        ArbolBinario<String> arbol = new ArbolBinario<>();
        Nodo<String> raiz = arbol.crearRaiz("PRUEBA");
        verificar(!arbol.esVacio(), "El árbol con raíz no debe estar vacío.");
        verificar(arbol.contarNodos() == 1, "El árbol debe tener un nodo.");
        verificar(arbol.contarHojas() == 1, "La raíz aislada debe ser una hoja.");
        verificar(arbol.altura() == 0, "La altura del árbol unitario debe ser cero.");
        verificar(raiz.esHoja(), "La raíz aislada debe reportarse como hoja.");
        verificar(raiz.grado() == 0, "La raíz aislada debe tener grado cero.");
    }

    private static ArbolBinario<String> construirArbolUniversitario() {
        ArbolBinario<String> arbol = new ArbolBinario<>();
        Nodo<String> universidad = arbol.crearRaiz("UNIVERSIDAD");
        Nodo<String> fisei = arbol.agregarIzquierdo(universidad, "FISEI");
        Nodo<String> dide = arbol.agregarDerecho(universidad, "DIDE");
        Nodo<String> software = arbol.agregarIzquierdo(fisei, "SOFTWARE");
        arbol.agregarDerecho(fisei, "TI");
        arbol.agregarIzquierdo(dide, "INVEST.");
        arbol.agregarDerecho(dide, "VINCULACIÓN");
        arbol.agregarIzquierdo(software, "NIVEL1");
        arbol.agregarDerecho(software, "NIVEL3");
        return arbol;
    }

    private static void probarArbolUniversitario(ArbolBinario<String> arbol) {
        verificar(arbol.contarNodos() == 9, "El árbol universitario debe tener nueve nodos.");
        verificar(arbol.contarHojas() == 5, "El árbol universitario debe tener cinco hojas.");
        verificar(arbol.altura() == 3, "El árbol universitario debe tener altura tres.");
        verificar(arbol.buscar("NIVEL1").esHoja(), "NIVEL1 debe ser una hoja.");
        verificar(arbol.buscar("NO EXISTE") == null, "Una búsqueda ausente debe devolver null.");
    }

    private static void probarInsercionOcupada(ArbolBinario<String> arbol) {
        Nodo<String> fisei = arbol.buscar("FISEI");
        int nodosAntes = arbol.contarNodos();
        boolean rechazada = false;
        try {
            arbol.agregarIzquierdo(fisei, "DUPLICADO");
        } catch (IllegalStateException e) {
            rechazada = true;
        }
        verificar(rechazada, "Debe rechazarse la inserción en una posición ocupada.");
        verificar(arbol.contarNodos() == nodosAntes, "El intento rechazado no debe modificar el árbol.");
    }

    private static void probarGrados(ArbolBinario<String> arbol) {
        verificar(arbol.buscar("FISEI").grado() == 2, "FISEI debe tener grado dos.");
        verificar(arbol.buscar("SOFTWARE").grado() == 2, "SOFTWARE debe tener grado dos.");
    }

    private static void verificar(boolean condicion, String mensaje) {
        if (!condicion) {
            throw new AssertionError(mensaje);
        }
    }
}
```